Task 1: Understanding Cyber Security Basics & Attack Surface

1. What is Cyber Security?

Cyber Security refers to the practice of protecting systems, networks, applications, and data from digital attacks, unauthorized access, damage, or disruption. These attacks often aim to steal sensitive information, disrupt services, or gain unauthorized control over systems.

Cyber security ensures trust, safety, and reliability in digital environments such as banking systems, social media platforms, e-commerce websites, and cloud services.

What does cybersecurity protect against?

- Hacking – breaking into systems without permission
- Malware – viruses, ransomware, spyware, etc.
- Phishing – fake emails or messages that steal information
- Data breaches – theft or exposure of sensitive data
- Denial-of-service attacks – shutting down websites or services

Key areas of cybersecurity

1. Network Security – protects networks from intruders
2. Application Security – secures software and apps
3. Information Security – protects data from unauthorized access
4. Cloud Security – safeguards cloud-based systems
5. Operational Security – manages how data is handled and protected
6. End-User Education – teaches people safe online behavior

Why is cybersecurity important?

- Protects personal information (passwords, bank details)
- Prevents financial loss
- Safeguards business operations
- Ensures privacy and trust
- Protects national security

Examples in everyday life

- Using strong passwords and two-factor authentication
- Antivirus and firewall protection
- Secure online banking
- Protecting social media accounts

2. CIA Triad (Core of Cyber Security)

The CIA Triad represents the three fundamental principles of cyber security:

a) Confidentiality

Ensures that sensitive information is accessible only to authorized users.

Examples:

- Bank account details protected using encryption
- WhatsApp end-to-end encrypted messages
- Login credentials hidden using password masking

Techniques Used:

- Encryption
- Authentication
- Access control

b) Integrity

Ensures that data is accurate, consistent, and not altered without authorization.

Examples:

- Transaction amounts not modified during online payments
- Software updates coming from trusted sources only

Techniques Used:

- Hashing
- Digital signatures
- Checksums

c) Availability

Ensures that systems and data are available when needed.

Examples:

- Banking apps accessible 24/7
- Websites protected against DDoS attacks

Techniques Used:

- Backups
- Redundancy
- Load balancing

3. Types of Cyber Attackers

Cyber attacks come in many forms, targeting systems, networks, and users. Here are the main types of cyber attacks, grouped for clarity:

Malware-Based Attacks

Malware is malicious software designed to harm or exploit systems.

- Virus – Attaches to files and spreads when executed
- Worm – Self-replicates across networks without user action
- Trojan Horse – Disguised as legitimate software
- Ransomware – Encrypts data and demands payment
- Spyware – Secretly monitors user activity
- Adware – Displays unwanted ads, sometimes tracking users
- Keylogger – Records keystrokes to steal credentials

Social Engineering Attacks

These exploit human behavior rather than technical vulnerabilities.

- Phishing – Fake emails or websites to steal data
- Spear Phishing – Targeted phishing aimed at specific individuals
- Whaling – Targets executives or high-level employees
- Vishing – Voice-based scams (phone calls)
- Smishing – SMS-based phishing
- Pretexting – Creating a fake scenario to gain trust

Network-Based Attacks

- Denial-of-Service (DoS) – Overwhelms a system to make it unavailable
- Distributed DoS (DDoS) – DoS using multiple compromised systems
- Man-in-the-Middle (MitM) – Intercepts communication between parties
- Packet Sniffing – Captures network traffic
- Session Hijacking – Takes over active user sessions

Web Application Attacks

- SQL Injection (SQLi) – Inserts malicious SQL queries
- Cross-Site Scripting (XSS) – Injects malicious scripts into websites
- Cross-Site Request Forgery (CSRF) – Forces users to perform unwanted actions
- File Inclusion (LFI/RFI) – Includes malicious files on a server

Credential & Access Attacks

- Brute Force Attack – Tries all possible passwords

- Dictionary Attack – Uses common passwords
- Credential Stuffing – Uses leaked credentials from other breaches
- Privilege Escalation – Gains higher access rights

Advanced & Specialized Attacks

- Zero-Day Attack – Exploits unknown vulnerabilities
- Advanced Persistent Threat (APT) – Long-term targeted attacks
- Supply Chain Attack – Compromises trusted software or vendors
- Insider Threat – Malicious or negligent employees
- Botnet Attack – Network of infected devices controlled remotely

Wireless & Mobile Attacks

- Evil Twin Attack – Fake Wi-Fi access point
- Bluejacking / Bluesnarfing – Bluetooth-based attacks
- SIM Swapping – Hijacking phone numbers

1. Script Kiddies

What is a Script Kiddie?

A script kiddie is an unskilled attacker who uses pre-written tools, scripts, or exploits created by others to launch attacks, without understanding how they work.

Key Characteristics:

- Low technical knowledge
- Uses ready-made hacking tools
- Motivated by curiosity, fun, or bragging rights
- Causes damage unintentionally at times

Common Attacks Used:

- Website defacement
- Simple DoS/DDoS attacks
- Basic password cracking

2. Insider Threats

An **insider threat** is a **security risk caused by a person inside an organization** who has **authorized access** to systems, data, or networks and **misuses that access**, intentionally or unintentionally.

Who Can Be an Insider?

- Employees
- Former employees

- Contractors
- Business partners or vendors

Types of Insider Threats

1. Malicious Insider

- Intentionally causes harm
- Steals data, sabotages systems, or commits fraud
- Example: An employee selling company data to competitors

2. Negligent (Careless) Insider

- No malicious intent
- Causes harm due to mistakes or poor security practices
- Example: Clicking a phishing link or using weak passwords

3. Compromised Insider

- Insider's account is taken over by an external attacker
- Appears legitimate but is controlled by hackers
- Example: Attacker uses stolen employee credentials

Common Insider Threat Activities

- Data theft or leakage
- Unauthorized data access
- Privilege abuse
- Installing unauthorized software
- Sharing login credentials

3. Hacktivists

A hacktivist is a hacker who conducts cyber attacks to promote political, social, or ideological causes.

Key Characteristics:

- Medium to high technical skills
- Motivation is ideology or activism
- Targets governments, corporations, or organizations
- Publicly claims responsibility for attacks

Common Attacks Used:

- Website defacement
- Data leaks (doxxing)

- DDoS attacks
- Social media account hijacking

4. Cyber Criminals

- Financial motivation
- Use phishing, ransomware, fraud

5. Nation-State Attackers

- Highly skilled government-backed hackers
- Perform espionage or cyber warfare
- Target critical infrastructure

4. What is an Attack Surface?

An **attack surface** is the **total number of points (entryways)** where an **attacker can try to gain unauthorized access** to a system, network, application, or organization.

Components of an Attack Surface

1. Digital Attack Surface

- Websites and web applications
- Open ports and services
- APIs
- Databases
- Cloud services
- User accounts and credentials

2. Physical Attack Surface

- Servers and computers
- USB ports and external devices
- Network equipment
- Access cards and physical entry points

3. Human Attack Surface

- Employees and users
- Social engineering targets
- Weak passwords
- Lack of security awareness

Examples

- An application with unnecessary open ports

- Employees using personal devices for work
- Public-facing APIs without authentication
- Outdated software with known vulnerabilities

5. OWASP Top 10 (Why It Is Important)

The OWASP Top 10 represents the most critical web application security risks, ranked by prevalence, detectability, and exploitability based on industry data. Each vulnerability includes a description, real-world example, and key impacts, drawn from the 2021 edition (still widely referenced, with 2025 updates emphasizing similar themes).

A01: Broken Access Control

Attackers bypass authorization to access unauthorized functionality or data, such as viewing or altering restricted resources. A classic example is the 2019 Capital One breach, where a misconfigured AWS role allowed an attacker to access 100 million customer records by exploiting insecure direct object references in URLs (e.g., changing /user/123 to /user/456). This tops the list due to its high incidence in 94% of tested applications, enabling massive data theft.

A02: Cryptographic Failures

Previously "Sensitive Data Exposure," this involves failures to properly protect sensitive data through encryption or hashing. The 2017 Equifax breach exposed 147 million records because the Apache Struts library lacked proper patching and encryption, allowing unencrypted SSN data to be stolen via intercepted traffic. Impacts include identity theft and compliance violations like GDPR fines.

A03: Injection

Untrusted input executes unintended commands, such as SQL, OS, or LDAP injection. The 2007 TJX Companies hack used SQL injection to steal 45 million credit card details by injecting payloads like ' OR 1=1-- into login forms, dumping entire databases. It remains critical as poor input validation affects over 8% of apps, leading to data breaches.

A04: Insecure Design

Flaws in application architecture from the start, like missing threat modeling. The Twitter (X) 2022 breach exploited design flaws in API rate limiting, allowing credential stuffing to hijack high-profile accounts for Bitcoin scams. This new category highlights how poor design enables scalable attacks.

A05: Security Misconfiguration

Improper setup of security settings, such as default credentials or exposed debug modes. The 2017 Uber breach occurred via an AWS bucket misconfiguration with admin access left public, leaking 57 million user records. Common in cloud environments, it affects 90% of apps.

A06: Vulnerable and Outdated Components

Using third-party libraries with known exploits without updates. Log4Shell (CVE-2021-44228) in Log4j 2021 impacted millions of apps, enabling remote code execution; e.g., exploited in Minecraft servers for ransomware. Dependency blindness causes widespread compromise.

A07: Identification and Authentication Failures

Weak session management or credential handling allows impersonation. The 2012 LinkedIn breach exposed 117 million unsalted SHA-1 hashed passwords, cracked via brute force for account takeovers. Credential stuffing exploits this in 20% of breaches.

A08: Software and Data Integrity Failures

Unverified code or updates from CI/CD pipelines. The 2020 SolarWinds attack injected malware into trusted software updates, compromising 18,000 organizations including U.S. agencies. A new entry reflecting supply chain risks.

A09: Security Logging and Monitoring Failures

Insufficient logging obscures breaches. The 2013 Target breach went undetected for weeks due to poor monitoring, allowing 40 million cards stolen before response. Delays amplify damage.

A10: Server-Side Request Forgery (SSRF)

Apps fetch remote resources without validation, accessing internal systems. The 2019 Capital One SSRF exploited a web app firewall misconfig to query AWS metadata, stealing 100 million records. Rare but devastating in cloud setups.

Map daily-used applications (email, WhatsApp, banking apps) to possible attack surfaces.

An **attack surface** refers to all the possible points where an attacker can interact with or attempt to compromise an application. When we look at **daily-used applications** such as email, WhatsApp, and banking apps, each of them exposes different attack surfaces because of how they are designed and how users interact with them.

Email applications have one of the largest attack surfaces because they are open communication platforms. The inbox itself becomes an entry point when users receive emails from unknown or spoofed senders. Attachments and links inside emails are common attack surfaces, as attackers can hide malware in files or redirect users to fake websites that steal login credentials. The login page of the email service is another attack surface, often targeted through phishing or brute-force attacks. In addition, password recovery features can be exploited if security questions are weak. For example, when a user receives an email claiming to be from their email provider asking them to “verify their account,” they may click a malicious link and unknowingly give their username and password to an attacker, allowing full access to their email.

Messaging applications such as WhatsApp have a different but equally significant attack surface. The primary attack surface is phone-number-based authentication, which relies on OTPs sent via SMS or calls. If an attacker tricks a user into sharing this OTP, the attacker can take over the account. Media files such as images, videos, and documents also form part of the attack surface, as specially crafted files can exploit vulnerabilities in the app or device. Group chats increase exposure because users are more likely to trust messages from known contacts. Cloud backups of chats are another attack surface if they are not strongly encrypted. A common example is when an attacker pretends to be a friend or WhatsApp support and asks for the OTP; once shared, the attacker gains access to all chats and contacts.

Banking and payment applications have a highly sensitive attack surface because they deal with financial data. The login interface, including PINs, passwords, and biometric authentication, is a major target for attackers. OTP mechanisms used for transaction verification are also vulnerable to social engineering and SIM-swapping attacks. The APIs that connect the mobile app to bank servers form a technical attack surface, especially if they are poorly secured. The user’s device and network connection are additional attack surfaces; malware on the phone or an insecure public Wi-Fi network can be used to intercept or manipulate data. For example, a user might download a fake banking app that looks legitimate, enter their login details, and unknowingly allow an attacker to transfer money from their account.

Mapping daily-used applications to their attack surfaces is important because it helps both users and organizations understand where the real risks lie. When people know how attacks happen, they become more cautious about clicking links, sharing OTPs, or installing apps from unknown sources. For organizations, this understanding allows them to design better security controls, reduce unnecessary exposure, and protect sensitive data more effectively. In cybersecurity, identifying and minimizing the attack surface is a key step in preventing real-world attacks.

Document how data flows from user → application → server → database.

Data flow in a typical application begins with the **user**, who interacts with the system through a client interface such as a web browser or a mobile application. When the user enters data—for example, login credentials, a message, or a transaction request—this information is first processed locally by the application. At this stage, basic validations may occur, such as checking whether required fields are filled or whether the input format is correct. Once the data is prepared, the application sends it securely over the network, usually using encrypted protocols such as HTTPS or TLS, to prevent interception during transmission.

The data then reaches the **application server**, which acts as the central processing unit of the system. The server receives the request and performs deeper validation and authentication checks. For example, during login, the server verifies whether the user exists and whether the provided credentials are valid. The server also enforces business logic, such as access control rules, transaction limits, or workflow conditions. At this point, the server determines whether it needs to retrieve existing data or store new data, and accordingly prepares a request to the database.

Next, the **database** layer handles data storage and retrieval. The server sends structured queries to the database, such as SQL or NoSQL queries, to read or write data. The database processes these queries and either fetches the requested information or updates its records. For example, in a banking application, the database may check the user's account balance, deduct an amount, and store the updated balance. The database then returns the result of the operation back to the server.

After receiving the response from the database, the **server** processes the results and formats them into a response suitable for the application. This may include generating a success message, an error notification, or structured data such as JSON. The server then sends this response back to the application through a secure communication channel.

Finally, the **application** presents the processed information to the **user** in a readable and usable form. For instance, the user may see a successful login message, a displayed account balance, or confirmation of a completed transaction. This completes one full data flow cycle. Throughout this entire process, security controls such as encryption, authentication, authorization, logging, and input validation are applied at each stage to protect data confidentiality, integrity, and availability.

where attacks can happen during this flow.

When data flows from the **user to the application, then to the server, and finally to the database**, attacks can occur at **each stage of this flow** if proper security controls are not in place.

At the **user level**, attacks usually exploit human behavior or the user's device. If a user's device is infected with malware, attackers can capture keystrokes, steal credentials, or manipulate input before it even reaches the application. Phishing attacks can trick users into entering sensitive data into fake applications or websites that look legitimate. Weak passwords or reused credentials also allow attackers to impersonate users and initiate malicious requests.

During the interaction with the **application layer**, attacks often target the application's interface and input handling. If the application does not properly validate or sanitize user input, attackers can inject malicious data such as SQL injection strings or cross-site scripting (XSS) payloads. Authentication and session management weaknesses can allow attackers to hijack active sessions or bypass login controls. Insecure APIs used by mobile or web apps can also be abused to send unauthorized requests directly to the backend.

As data travels from the **application to the server over the network**, attacks can occur during transmission. If encryption is not used or is misconfigured, attackers can intercept or modify data through man-in-the-middle (MITM) attacks, especially on public or compromised networks. Replay attacks can occur if attackers capture valid requests and resend them later to repeat transactions or actions.

At the **server level**, attackers focus on exploiting vulnerabilities in server software, configurations, or business logic. Unpatched servers may be compromised through known exploits, while weak access controls can allow privilege escalation. Attackers may also perform denial-of-service (DoS) attacks to overwhelm the server and disrupt availability. Improper error handling can leak sensitive information, such as system details or stack traces, which helps attackers plan further attacks.

When the server communicates with the **database**, attacks typically target how queries are constructed and executed. If the server directly passes user input into database queries, attackers can perform SQL injection to read, modify, or delete sensitive data. Weak database authentication, excessive privileges, or exposed database ports can allow direct unauthorized access. In some cases, attackers may extract entire datasets or alter critical records.

Finally, during the **response flow back to the application and user**, attacks can still occur. Sensitive data returned without proper filtering or encryption can be exposed. Stored or reflected XSS attacks can execute malicious scripts in the user's browser when the response is rendered. If responses are not properly validated, attackers can manipulate them to mislead users or hide malicious activity.

In summary, attacks can happen **at every point in the data flow**, from the user's device to the database and back. Understanding where these attacks occur helps security teams apply controls such as strong authentication, input validation, encryption, access control, and monitoring at each stage of the system.